1

I apologize I'm going to do this problem with the voice recording or voice to text I should say really bad headaches so try not to look at screens right now but at least my approach to this problem is given our tree with Kay we know that we can rearrange the tree to fill all the vertices for each bag in our tree with k+1 vertices and we can also make it such that between two adjacent nodes we only differ by one vertex by replacing one for a next essentially so my approach to this problem would be starting at the roof the root node we essentially perform different traversals down the tree and looking at kind of each layer we're looking at when a node appears for the first time in our traversal and it should end potentially different parts in the tree but that's the gist of it so for each of the A's in our R vertex disjoint paths the first time they appear we label AI as I and then Traverse down the tree

we know that as we Traverse down the tree that we need to have at some point another vertex labeled eye in the same bag as our original for a text labeled I and so not sure what the runtime of this is you pick an arbitrary vertex and one of the bags and label it I and you iterate through every possible I label for each vertex until you reach your a bag that contains a Vertex already labeled I and your final be for that individual pair and of course that each time you label an individual vertex I you check to make sure there exists an edge between a previous node labeled I and this one

we know that we don't assign the same vertex to different numbers because once a Vertex is assigned a number and a bag it would be a sign that number for all bags we know this works because as we do a BFS traversal the first time we see a vertex is going to be the first time in every sort of Branch just by the definition of how the vertices come up in the tree and so we never assign the same vertex to distinct pads so essentially the number we assigned to a vertices is which path it's on and essentially also would then assign some vertices like a zero or something to indicate not part of any path maybe negative one

2

H is a topological minor of G if a subgraph of G can be constructed by only adding vertices on an existing edge in H.

Since H is fixed, ill use the notation that $|H| = k$ and $|G| = n$, let p be the runtime of the r-Vertex disjoint paths problem

Suppose the endpoints in our r-Vertex disjoint paths algorithm do not need to be distinct, ie a_i and a_j can have the same value. Then this problem is simply iterate over the $\binom{n}{k} = O(n^k)$ projections of the vertices in H onto G and let $r = |E(H)|$ and the pairs a_i, b_i be the edges in H. We simply run our r-Vertex disjoint algorithm on the projected pairs and if the algorithm returns true then we say that H is a topological minor of G in runtime $O(pn^k)$.

However if the endpoints in our r-Vertex disjoint paths algorithm need to be distinct, $i \neq j \rightarrow a_i \neq a_j$ then we need to make some slight modifications to G. The idea is still the exact same but now for scenarios when $a_i = a_j = v$ then replace v with v_i, v_j that are both connected to all the neighbors of v. Note that there is the edge case of when the neighbors of v contain some other a_l or b_l that also gets replaced in which case we would need to connect v_i and v_j to all the new duplicate vertices corresponding to the a_l or b_l or whatever. Note at worst case this increases n to be size kn which is still polynomial. Thus our final runtime is something like $O(p(kn)^k)$ which is polynomial.

Proof of correctness

H is a topological minor of G show our algorithm returns true

> If H is a topological minor of G then there is a subgraph of G that can be constructed by only adding vertices to existing edge in H. In other words we are able to form H from contracting along vertex disjoint paths in G. Since H is a topological minor, then there would exist these set of disjoint paths that are formed from the edges of H so our algorithm would return true because the paths exist and our r-Vertex disjoint paths sub algorithm would find them for some projection.

Algorithm returns true so show H is a topological minor of G

> If our algorithm returns true then that means we have these vertex disjoint paths in the same orientation as the edges of H. This means we would be able to contract along these paths until we have only the edges for a_i, b_i and if we remove all other vertices in our contracted G then we would be left with a graph that would be exactly isomporphic to H through whatever projection we found in the algorithm. Note that if we are doing the split algorithm then we would need to merge the v_i, v_j vertices back together to get the H. Regardless, we would then find that H is a topological minor of G

H is not a topological minor of G show our algorithm returns false

> If H is not a topological minor of G then there is no such subgraph of G that can be formed from adding vertices along edges of H. This means that there is no such way to form a subgraph of G by adding vertices to edges in H. This means that we would not be able to create a projection that has the same vertex disjoint paths as edges in H. Thus our algorithm would return false

Algorithm returns false so H is not a topological minor of G

> If the algorithm returns false that means in every projection of H onto G we are unable to find our r-Vertex disjoint paths in G. This means we are unable to find a paths subgraph of G where the paths correspond to the edges in H. This means

that there is no subgraph of G that can be formed by adding vertices along edges of H.